

Week 3 Assessment Summary

Muskan

oday

Overview

This document summarizes the completed hands-on exercises for the **Spring REST using Spring Boot** Deepskilling module, covering all five hands-on documents:

1. 1. spring-rest-handson.docx — Spring Boot project setup, XML bean configuration, logging
2. 2. spring-rest-handson.docx — Hello World REST service, HTTP request/response, MockMvc testing
3. 3. spring-rest-handson.docx — Employee & Department REST services backed by XML config
4. 4. spring-rest-handson.docx — POST/PUT/DELETE with bean validation and global exception handling
5. 5. JWT-handson.docx — Spring Security + JWT authentication/authorization

All solutions live in a single Spring Boot 3 (Java 17) Maven project: **spring-learn**.

Hands-on 1 — Project Setup

- Created via Spring Initializr equivalent: `com.cognizant:spring-learn`, Spring Web + DevTools.
- `SpringLearnApplication` — `@SpringBootApplication`, logs “Start”/“End” around `SpringApplication.run()`.
- `date-format.xml` — classic Spring bean XML defining a `SimpleDateFormat("dd/MM/yyyy")` bean, loaded via `ClassPathXmlApplicationContext` and used to parse “31/12/2018”.
- Logging configured through `application.properties` (`logging.level.*`, `logging.pattern.console`).

Hands-on 2 — Hello World REST Service + Testing

- `HelloController` exposes GET `/hello`, returning the literal string `Hello World!!`, with start/end log statements as required.
- `HelloControllerTest` uses `@AutoConfigureMockMvc` / `MockMvc` to call the endpoint and assert `status().isOk()` and the response body — an end-to-end test executable from Maven (`mvn test`) or an IDE, per the hands-on’s testing objective.

Hands-on 3 — Employee & Department Services

- `employee.xml` defines `Department` and `Employee` beans (2 departments, 4 employees) and two `util:list` collections (`employeeList`, `departmentList`), matching the “create static employee list data using Spring XML configuration” requirement.
- `EmployeeDao` / `DepartmentDao` load their respective lists from the XML config in a static initializer and expose `getAllEmployees()` / `getAllDepartments()`.
- `EmployeeService` (annotated `@Service`, method annotated `@Transactional`) and `DepartmentService` delegate to the DAOs.

- EmployeeController (GET /employees) and DepartmentController (GET /departments) expose the data as REST resources, ready to be consumed by the Angular employee-list screen described in the hands-on.

Hands-on 4 — Validation, HTTP Methods, Exception Handling

- Country model uses @NotBlank / @Size (Hibernate Validator) on code and name.
- CountryController follows the REST naming conventions taught in the hands-on:
 - GET /countries and GET /countries/{code}
 - POST /countries and PUT /countries with @Valid @RequestBody
 - DELETE /countries/{code}
- GlobalExceptionHandler (@RestControllerAdvice) centralizes error handling:
 - MethodArgumentNotValidException → 400 with a list of field-level validation messages
 - MethodArgumentTypeMismatchException → 400 (covers “number formatting errors”)
 - HttpMessageNotReadableException → 400 for malformed JSON
 - ResourceNotFoundException (custom) → 404
 - any other exception → 500
- CountryControllerTest verifies the success path, a validation failure (blank country name), and a not-found lookup.

Hands-on 5 — Spring Security + JWT

- pom.xml adds spring-boot-starter-security and io.jsonwebtoken (jjwt-api / jjwt-impl / jjwt-jackson).
- SecurityConfig defines a SecurityFilterChain bean (the modern replacement for the deprecated WebSecurityConfigurerAdapter shown in the original hands-on): stateless sessions, CSRF disabled, /auth/login and /hello public, /countries/** requiring role USER or ADMIN.
- AuthController (POST /auth/login) authenticates the two in-memory demo accounts from the hands-on (admin / user, password pwd) and issues a signed JWT containing the user’s role.
- JwtUtil creates and validates HS256-signed JWTs (subject, role claim, expiry).
- JwtAuthFilter (a OncePerRequestFilter) reads the Authorization: Bearer <token> header on every request, validates the token, and populates Spring Security’s SecurityContext with a ROLE_<role> authority — reproducing the “authorize requests using Spring Filters” objective.

How to run

```
cd spring-learn
mvn clean package
mvn spring-boot:run           # starts on port 8090
mvn test                      # runs HelloControllerTest +
CountryControllerTest
```

Sample requests

```
curl http://localhost:8090/hello
curl http://localhost:8090/employees
curl http://localhost:8090/departments

curl -s -X POST http://localhost:8090/auth/login \
  -H "Content-Type: application/json" -d
'{"username":"user","password":"pwd"}'
```

```
curl -H "Authorization: Bearer <TOKEN>"  
http://localhost:8090/countries
```